

Cx1106
Computer
Organisation and
Architecture
**Computer
Arithmetic**

Prof Lin WeiSi
School of Computer Science and
Engineering, NTU
Email: wslin@ntu.edu.sg

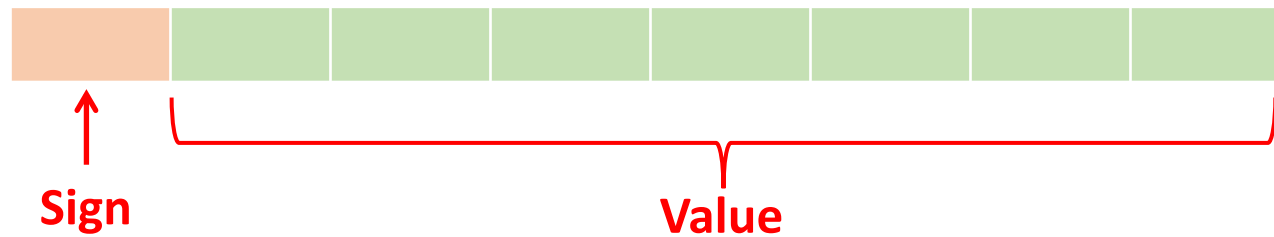


Binary numbers are the basis for all data representation in digital computer systems.

Why binary?

Positive and Negative Numbers

- The content below will focus mainly on the binary system.
- To represent **signed integers**, computer systems allocate the **Most Significant Bit (MSB)** to indicate the **sign** of a number
 - **MSB = 0** indicates a **positive** number
 - **MSB = 1** indicates a **negative** number
- Remaining bits contain the value of the number, which can be interpreted in different ways



- Signed binary integers can be expressed using different number formats, and for this course, we will touch on the most commonly used format in computing
 - **Two's Complement Representation**

Two's Complement Representation

- Positive numbers are represented as normal binary
- Negative numbers are created by negation
 - Invert all bits of the number (flip the bits)
 - Add one to the inverted result (ignoring any overflow)

Example:

+106	0	1	1	0	1	0	1	0
-------------	---	---	---	---	---	---	---	---

+106	0	1	1	0	1	0	1	0
-------------	---	---	---	---	---	---	---	---

Invert	1	0	0	1	0	1	0	1
---------------	---	---	---	---	---	---	---	---

Add 1	0	0	0	0	0	0	0	1
--------------	---	---	---	---	---	---	---	---

-106	1	0	0	1	0	1	1	0
-------------	---	---	---	---	---	---	---	---

- Most Significant Bit (MSB) indicates the sign of a number
 - MSB = 0 indicates a positive number
 - MSB = 1 indicates a negative number



Two's Complement To Decimal Conversion

Example: What is the decimal representation for 10001110_2

Table of weights

-2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
-128	64	32	16	8	4	2	1
1	0	0	0	1	1	1	0
-128	0	0	0	8	4	2	0

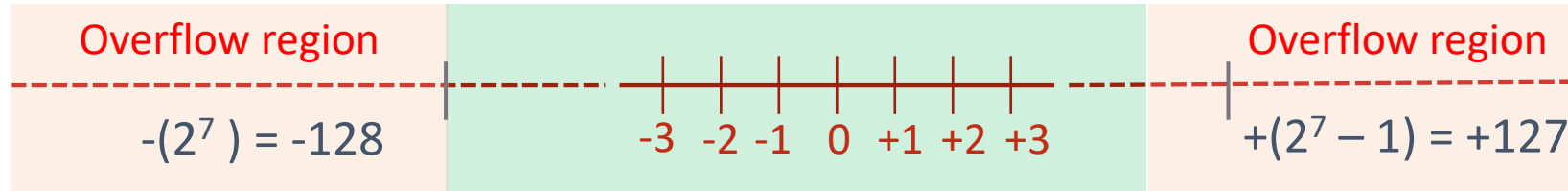
$$\textcolor{red}{-128 + 8 + 4 + 2 = -114}$$

The decimal representation for $10001110_2 = -114$



Carry vs Overflow Example

(illustrated with 8-bit representation)



Example: $48 - 19 = 48 + (-19) = 29$

First, negate 19 (00010011 $\rightarrow$ 11101101)

Then add the two numbers and discard any carries emitting from the high order bit.

		1	1					
	0	0	1	1	0	0	0	0
+	1	1	1	0	1	1	0	1
	0	0	0	1	1	1	0	1

- Carry does not always mean that we have an error/overflow
- So how do we detect overflow?

Detecting Overflow in Two's Complement Numbers

- **Overflow** can be easily detected by checking the **Most Significant Bit (MSB)** of the operands and result
- **Conditions for overflow**
 - In **addition** ($\text{Result} = A + B$)
 - If $\text{MSB}(A) = \text{MSB}(B)$, and $\text{MSB}(\text{Result}) \neq \text{MSB}(A)$
 - In **subtraction** ($\text{Result} = A - B$)
 - If $\text{MSB}(A) \neq \text{MSB}(B)$ and $\text{MSB}(\text{Result}) \neq \text{MSB}(A)$



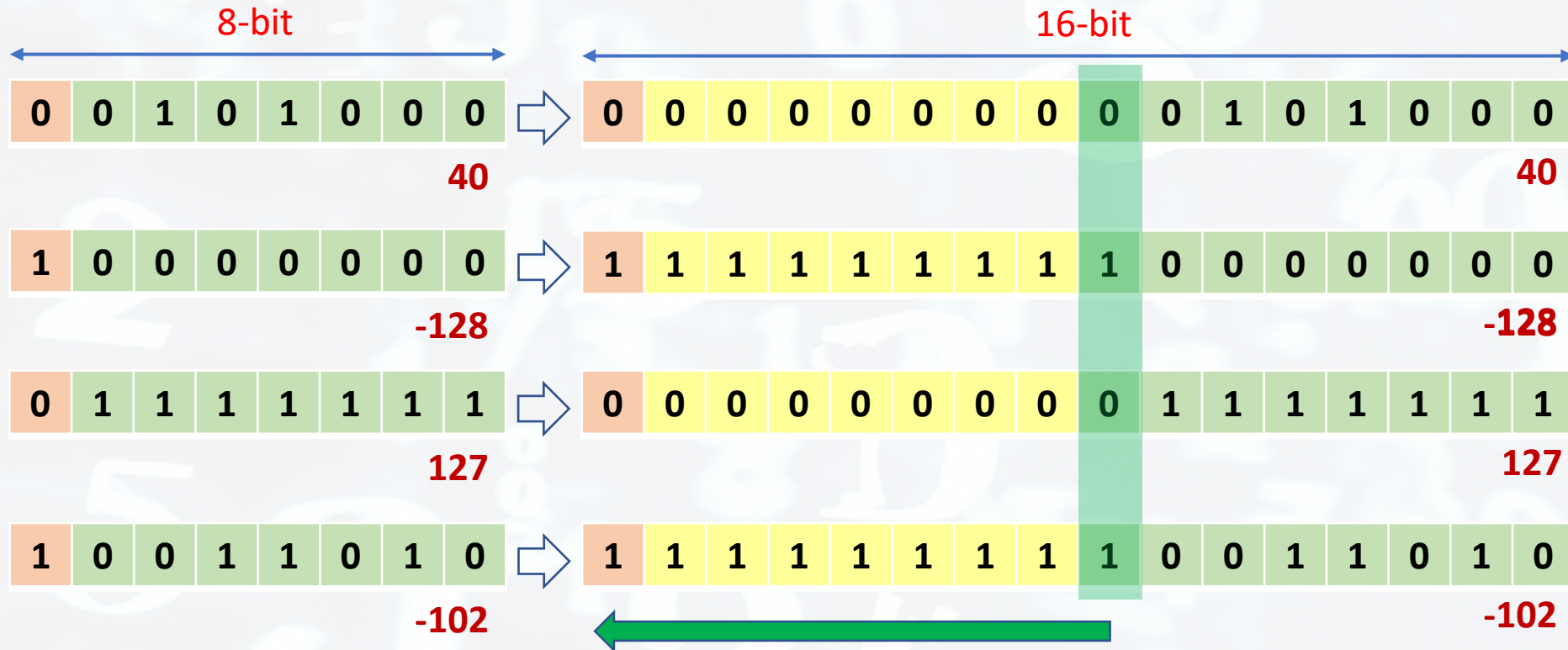
Operation	Conditions		Result
$A + B$	$A > 0$	$B > 0$	< 0
$A + B$	$A < 0$	$B < 0$	> 0
$A - B$	$A > 0$	$B < 0$	< 0
$A - B$	$A < 0$	$B > 0$	> 0

→ overflow

-

Sign Extension

- In two's complement, sign extension is needed to **convert a smaller size operand to a larger size operand**



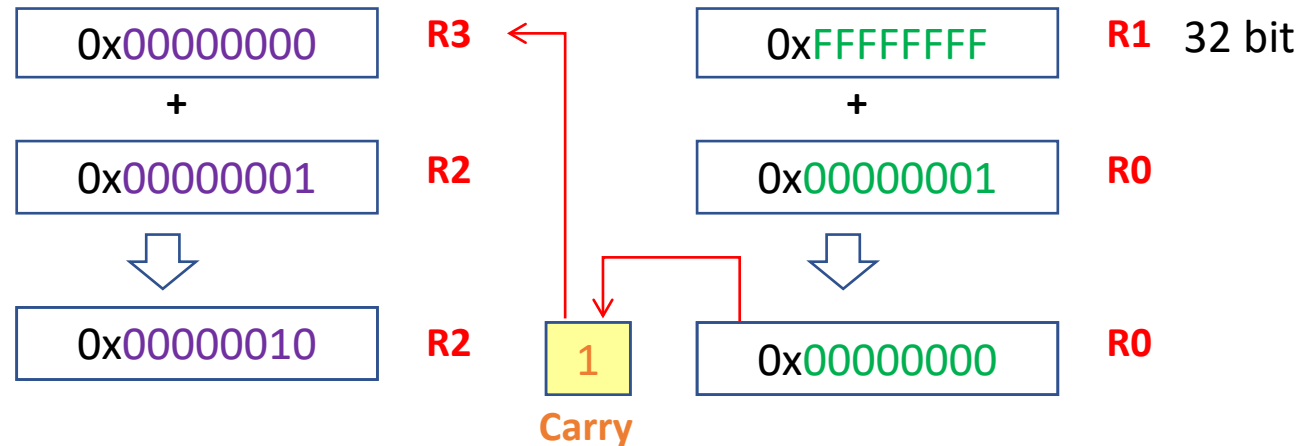
- Sign extension simply **copies the sign bit (MSB) into the higher order bits**

Multi-Precision Arithmetic

- How can we add operands that are larger than 32-bits (e.g. 64 bit operands) if we have only a single 32-bit ALU?
- The solution is to reuse the 32-bit adder for multi-precision addition
- Multi-precision arithmetic involves the computation of numbers whose precision is larger than what is supported by the maximum size of the processor register (Single-Precision)

```
00000000 FFFFFFFF
+ 00000001 00000001
-----
00000010 00000000
```

```
ADD  R0 , R0 , R1 ; add lower word with carry out
ADC  R2 , R2 , R3 ; add upper word with carry in
```





Cx1106

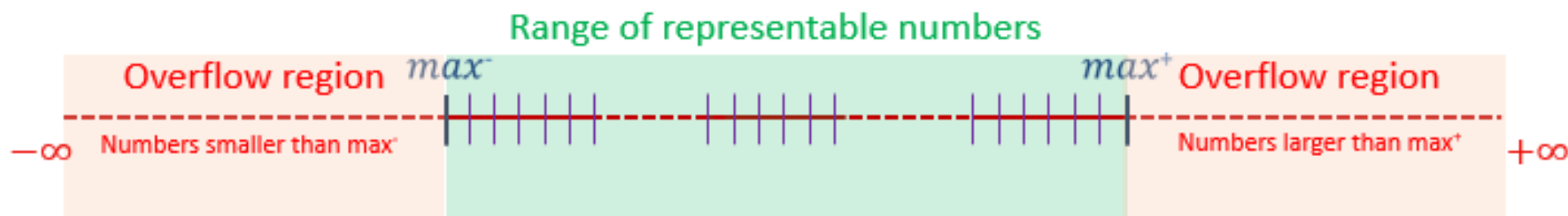
Computer Organisation and Architecture

Fixed and Floating Point Number System

Prof Lin WeiSi
School of Computer Science and
Engineering, NTU
Email: wslin@ntu.edu.sg

Range and Precision

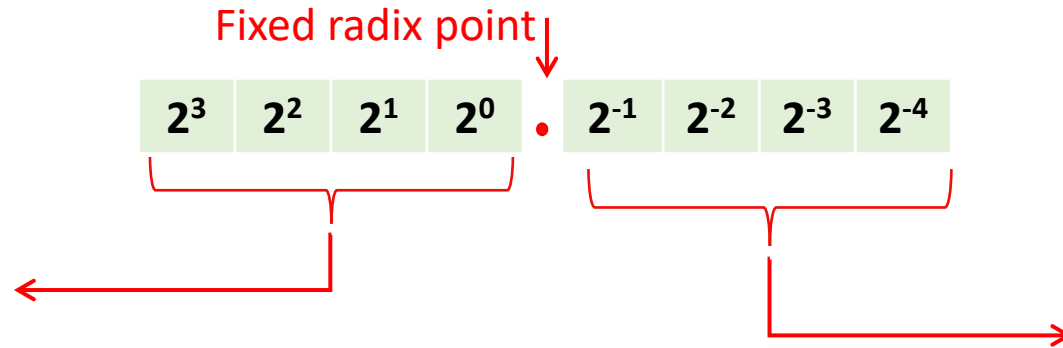
- **Range:** Interval between smallest ($\max^-$) and largest ($\max^+$) representable number
 - Example: Range of two's complement is $-(2^{(N-1)})$ to $(2^{(N-1)}-1)$
 - Each tick mark is a representable number in the range
- **Precision:** Amount of information used to represent each number
 - Example: 1.666 has higher precision than 1.67
 - The number of tick marks provides an indication of precision



Fixed-Point Representation

- Fixed-point format can represent **integer** and/or **fractional** values

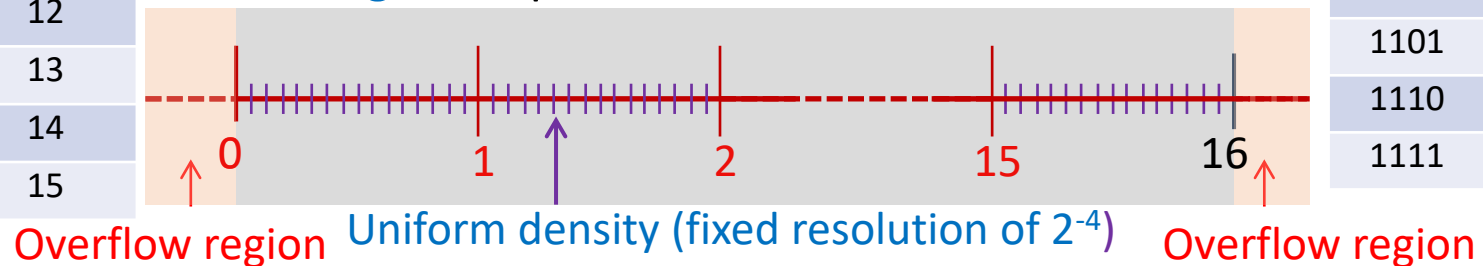
Unsigned Integer	
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15



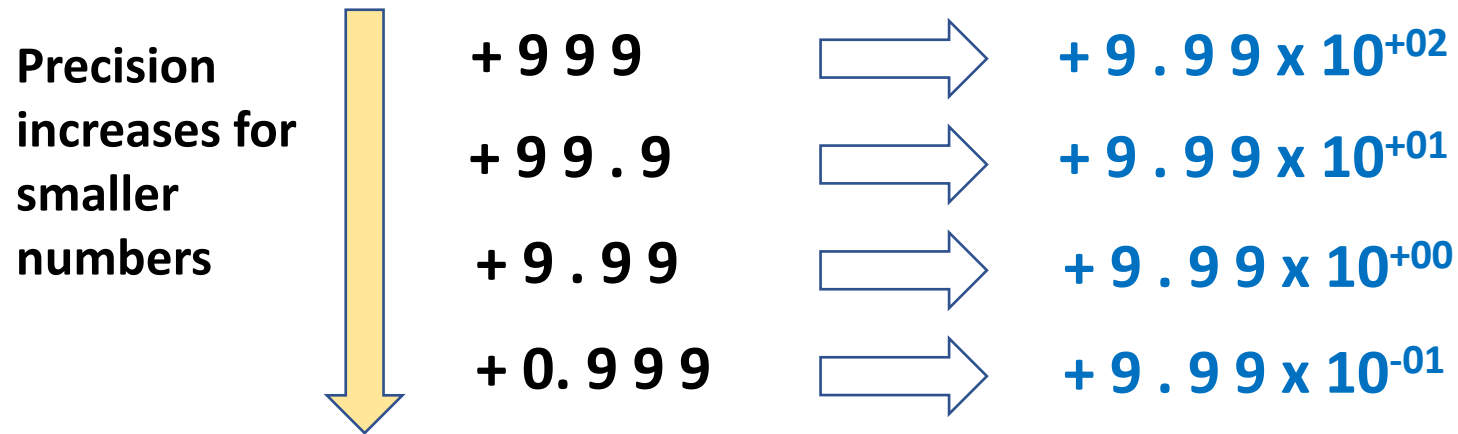
- Precision is limited by the range of integer values
- allocate more bits for integer → the range increases but the precision reduces.

Fractional Values	
0000	0.0000
0001	0.0625
0010	0.1250
0011	0.1875
0100	0.2500
0101	0.3125
0110	0.3750
0111	0.4375
1000	0.5000
1001	0.5625
1010	0.6250
1011	0.6875
1100	0.7500
1101	0.8125
1110	0.8750
1111	0.9375

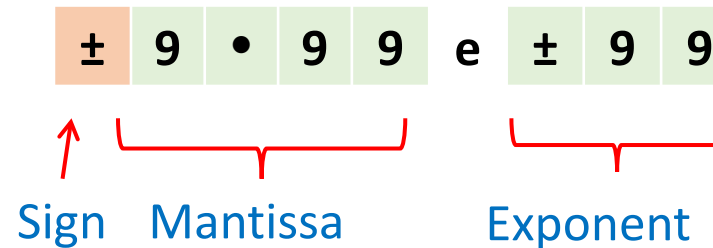
Range of representable numbers



Floating Point Representation

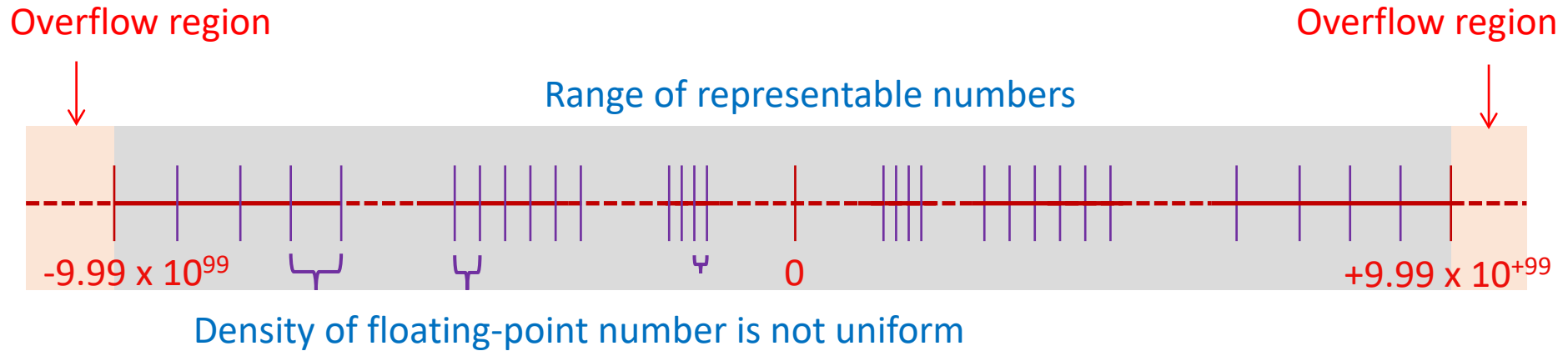


Simple decimal (Radix = 10)
floating point format



- Three main fields needed for floating-point representation:
 - **Sign** – denote positive/negative number
 - **Mantissa** – base value
 - **Exponent** – specifies position of radix point

Floating Point Representation



- Floating point representation can represent values across a wide range (-9.99×10^{99} to $+9.99 \times 10^{99}$).
- Size of **exponent** determines **range** of representable numbers.
- Size of **mantissa** and **value of exponent** determines the representable **precision**.
- **Small numbers** can be represented with **good precision**. When representing **large numbers**, precision can be sacrificed to achieve **greater range**.

Normalisation

- In the simple decimal floating-point format, there **are multiple representations** for the same value

±	1	•	0	0	e	±	0	1	Normalised
±	0	•	1	0	e	±	0	2	
±	0	•	0	1	e	±	0	3	

- **Normalisation** is necessary to **avoid synonymous representation** by maintaining **one non-zero digit before the radix-point**
 - In decimal number, this digit can be from 1 to 9
 - In binary number, this digit should be 1
- Normalisation can **maximise** number of bits of **precision**

±	5	•	4	2	e	±	0	3	Normalised
±	0	•	5	4	e	±	0	4	

Underflow

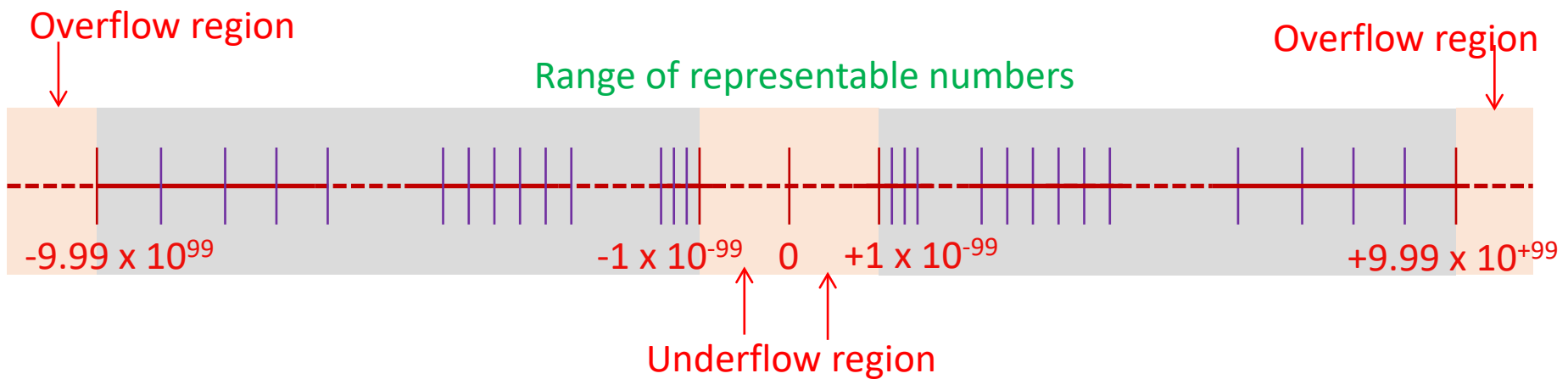
- Normalisation results in **underflow** regions where **values close to zero cannot be represented**

Smallest positive normalised number

+	1	•	0	0	e	-	9	9
---	---	---	---	---	---	---	---	---

Smallest negative normalised number

-	1	•	0	0	e	-	9	9
---	---	---	---	---	---	---	---	---



- Underflow occurs when a value is too small to be represented
- Floating-point **overflow and underflow** can cause programs to crash if not handled properly.

The background of the slide is a complex, abstract composition. It features a grid of numbers (0-9) and letters (A-Z) in various colors (blue, green, yellow, orange) and sizes, overlaid on a pattern of translucent, overlapping geometric shapes like triangles and squares. The overall effect is a sense of digital complexity and data flow.

Cx1106

Computer Organisation and Architecture

IEEE754

Prof Lin WeiSi
School of Computer Science and
Engineering, NTU
Email: wslin@ntu.edu.sg

IEEE 754 Floating Point Standard

- Found in virtually every computer since 1980
 - Simplified porting of floating-point numbers
 - Unified the development of floating-point algorithms

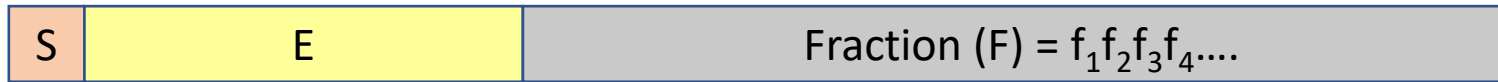
- Single Precision Floating-Point Numbers (32-bits)
 - 1-bit sign + 8-bit exponent + 23-bit fraction



- Double Precision Floating-Point Numbers (64-bits)
 - 1-bit sign + 11-bit exponent + 52-bit fraction



IEEE 754 Normalised Numbers



$$(-1)^S \times (1.F)_2 \times 2^{E - \text{Bias}}$$

■ Sign bit

- S = 0 (positive); S = 1 (negative)

■ Exponent

- Biased representation (0000 0001 to 1111 1110)
- Value of **exponent** = E - **Bias**
- **Bias** = **127** (Single Precision) and **1023** (Double Precision)

■ Fraction

- Assumes **hidden 1.** (not stored) for normalised numbers
- Value of normalised floating point number is:
$$(-1)^S \times (1 + f_1 \times 2^{-1} + f_2 \times 2^{-2} + f_3 \times 2^{-3} + f_4 \times 2^{-4} + \dots)_2 \times 2^{E - \text{Bias}}$$



Converting Single Precision To Decimal

- Find the decimal value of these single precision number:

0 1 0 1 1 0 0 1 0 1 1 1 0

Sign = 0 (positive)

Exponent = $10110010_2 = 178$; E - Bias = $178 - 127 = 51$

1 + Fraction = $(1.111)_2 = 1 + 2^{-1} + 2^{-2} + 2^{-3} = 1.875$

Value in decimal = $+1.875 \times 2^{51}$

1 0 0 0 0 1 1 0 0 0 1 0 1 0

Sign = 1 (negative)

Exponent = $00001100_2 = 12$; E - Bias = $12 - 127 = -115$

1 + Fraction = $(1.0101)_2 = 1 + 2^{-2} + 2^{-4} = 1.3125$

Value in decimal = -1.3125×2^{-115}



Representable Range for Normalised Single Precision

- In **normalised mode**, exponent is from 00000001 to 11111110
- Smallest magnitude** normalised number

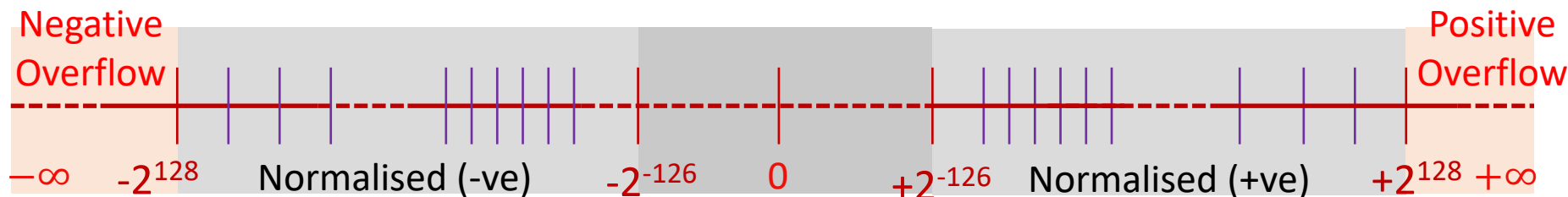
X 0 0 0 0 0 0 0 0 1 0

Exponent = $(00000001)_2 = 1$; E - Bias = $1 - 127 = -126$
 1 + Fraction = $(1.000...000)_2 = 1$
 Value in decimal = 1×2^{-126}

- Largest magnitude** normalised number

X 1 1 1 1 1 1 1 0 1

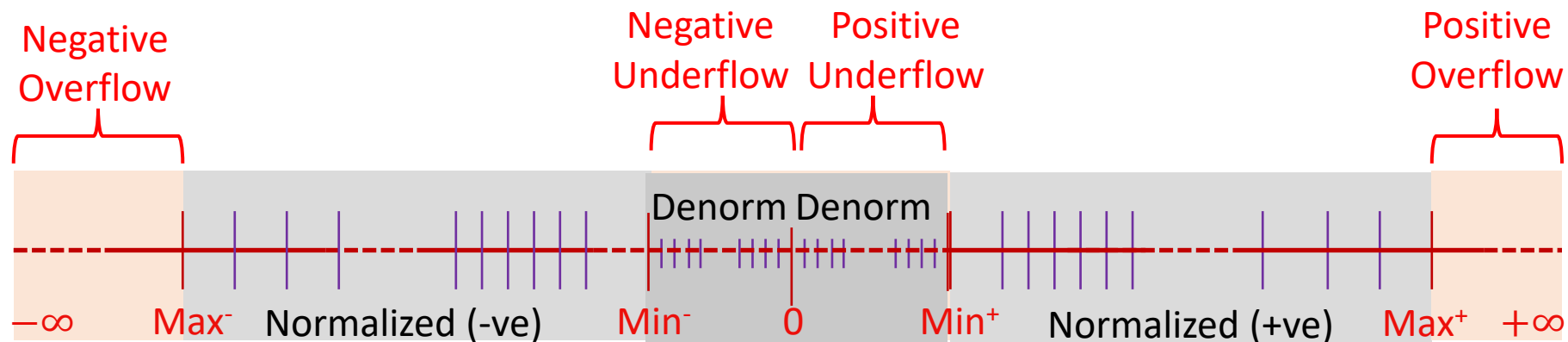
Exponent = $(11111110)_2 = 254$; E - Bias = $254 - 127 = 127$
 1 + Fraction = $(1.111...111)_2 \approx 2$
 Value in decimal $\approx 2 \times 2^{127} = 2^{128}$



IEEE 754 Encoding



Mode	Sign	Exponent	Fraction
Normalized	1 / 0	00000001 to 11111110	Anything
Denormalized	1 / 0	00000000	Non zero
Zero	1 / 0	00000000	0000 ... 0000
Infinity	1 / 0	11111111	0000 ... 0000
Not a Number (NaN)	1 / 0	11111111	Non zero



1400000
1100000
7100000
001000
7040010
750104
100100
100770
61411/
441417
779616

Fixed Point vs Floating Point Number System

- Given the same number of bits to represent a data, e.g. 32 bits.
- Floating Point (IEEE754)
 - Max Range $\approx 2 * 2^{128}$ (-2^{128} to $\sim 2^{128}$).
 - Max Precision (near to zero) less than 2^{-126}
- Fixed Point
 - Max Range (Radix right of LSB, unsigned) $\approx 2^{32}$
 - Max Precision (Radix left of MSB) 2^{-32}
- Floating point yield a larger range and better precision at small numbers with the same number of bits representation.
 - One usually needs the best precision when the numbers are small.
- However, Fixed point number has the advantage of having uniform precision across entire range.
 - Floating point number's precision changes across the range and the very coarse precision at the two end of the range may not be desirable to an intended algorithm.



No part of this material shall be filmed, recorded, downloaded, reproduced, distributed, republished, or transmitted in any form or by any means without written approval from Nanyang Technological University.